

Multi-Region AWS VPC Peering

with EC2, Apache & Terraform

Complete Project Documentation — Step-by-Step Guide & Network Flow

Author	Ritik
Primary Region	us-east-1 (VPC CIDR: 10.0.0.0/16)
Secondary Region	us-west-2 (VPC CIDR: 10.1.0.0/16)
Instance Type	t3.micro — Amazon Linux 2023
Web Server	Apache HTTPD (installed via user_data)
Connectivity	VPC Peering — cross-region, private
IaC Tool	Terraform ~> AWS Provider 6.43.0
Date	22-05-2026

1. Project Overview

This project uses Terraform to provision a fully connected, dual-region AWS infrastructure. Two EC2 instances — one in us-east-1 (primary) and one in us-west-2 (secondary) — are deployed inside their own VPCs and connected via VPC Peering, enabling private cross-region communication without traversing the public internet.

Key highlights:

- Dual-region deployment — separate AWS providers using Terraform aliases
- Cross-region VPC Peering — bidirectional private routing between 10.0.0.0/16 and 10.1.0.0/16
- Dynamic data sources — AZs and latest Amazon Linux 2023 AMI fetched automatically
- SSH key pairs — generated per region, registered via AWS EC2 key-pair API
- Apache web server — auto-installed on boot via user_data shell scripts
- Security groups — SSH (22), HTTP (80), ICMP (ping) open; all egress allowed
- Outputs — public IPs of both instances printed after terraform apply

2. Project File Structure

[Diagram rendering unavailable: Command failed: python3 -c "import cairosvg; cairosvg.svg2png(url='/tmp/diag_1779541283789.svg', write_to='/tmp/diag_1779541283789.png', output_width=700, output_height=340)" 2>/dev/null]

3. Architecture Diagram

The diagram below shows both VPCs, their subnets, internet gateways, route tables, security groups, EC2 instances, and the VPC peering connection between the two regions.

[Diagram rendering unavailable: Command failed: python3 -c "import cairosvg; cairosvg.svg2png(url='/tmp/diag_1779541283531.svg', write_to='/tmp/diag_1779541283531.png', output_width=900, output_height=520)" 2>/dev/null]

4. Complete Network Flow & Why Each Resource Exists

Every resource in this project plays a specific role in the network. The table below explains what each resource does and why it is needed:

Resource	What It Does	Why It Is Needed
<code>aws_vpc (×2)</code>	Creates an isolated virtual network in each region	Without a VPC, you cannot launch EC2 instances or control networking. Each region gets its own non-overlapping CIDR (10.0/16 and 10.1/16) so peering routes are unambiguous.
<code>enable_dns_hostnames / dns_support</code>	Makes VPC resolve DNS hostnames	Needed so EC2 instances can resolve each other and AWS service endpoints by name inside the VPC.
<code>aws_subnet (×2)</code>	Carves a slice of the VPC CIDR for instances	Instances live in subnets, not directly in VPCs. <code>map_public_ip_on_launch = true</code> gives each EC2 a public IP automatically so you can SSH in from the internet.
<code>data.aws_availability_zones (×2)</code>	Fetches available AZs per region at plan time	Avoids hardcoding AZ names. Uses <code>names[0]</code> dynamically so the code works regardless of which AZs exist in the region.
<code>data.aws_ami (×2)</code>	Fetches latest Amazon Linux 2023 AMI per region	AMI IDs differ per region and change with updates. A data source always resolves the most recent one so you never use an outdated AMI.
<code>aws_internet_gateway (×2)</code>	Provides a route to/from the public internet	Without an IGW, instances in public subnets cannot send or receive internet traffic. Required for HTTP access and outbound yum installs during boot.
<code>aws_route_table (×2)</code>	Defines where traffic goes based on destination CIDR	Default VPC route tables have no internet route. Custom route tables add: <code>0.0.0.0/0 → IGW (internet)</code> and <code>10.x.0.0/16 → VPC Peering (private cross-region)</code> .

<code>aws_route_table_association (×2)</code>	Binds the custom route table to the subnet	Without association, the subnet uses the default route table. The association makes the custom routes (including peering) active for instances in that subnet.
<code>aws_vpc_peering_connection</code>	Sends a peering request from primary to secondary	Opens the cross-region private channel between the two VPCs. <code>auto_accept=false</code> because the acceptor is in a different region/provider.
<code>aws_vpc_peering_connection_accepter</code>	Accepts the peering request in the secondary region	Cross-region peering requires explicit acceptance in the peer region. The secondary provider accepts automatically (<code>auto_accept=true</code>) via Terraform.
<code>aws_route (primary→secondary)</code>	Adds a route: 10.1.0.0/16 → peering connection	Without this route, packets from the primary EC2 destined for 10.1.x.x have no path and are dropped. <code>depends_on</code> ensures peering is accepted first.
<code>aws_route (secondary→primary)</code>	Adds a route: 10.0.0.0/16 → peering connection	Return path. Without it, packets from secondary reach primary but replies cannot get back — peering must be bidirectional in both route tables.
<code>aws_security_group (×2)</code>	Stateful firewall per instance	Controls which traffic is allowed in and out. Ingress: port 22 (SSH), port 80 (HTTP), ICMP (ping — needed to verify connectivity). Egress: all allowed.
<code>aws_key_pair (primary/secondary)</code>	Registers SSH public key with AWS per region	Allows password-free SSH login to EC2 instances. Each region requires its own key pair because key pairs are regional resources.
<code>aws_instance (×2)</code>	Launches the EC2 virtual machine	The actual compute resource. <code>user_data</code> runs the shell script on first boot to install Apache. <code>key_name</code> attaches the SSH key for secure access.
<code>primary.sh / secondary.sh</code>	Shell scripts run as <code>user_data</code> on EC2 boot	Automates Apache installation (yum install httpd), enables it to start on reboot, and writes a unique <code>index.html</code> to identify which region is serving.

5. Network Traffic Flow Diagram

This diagram illustrates how traffic flows from the internet to each EC2 instance, and how the two instances communicate privately via VPC Peering.

[Diagram rendering unavailable: Command failed: python3 -c "import cairosvg; cairosvg.svg2png(url='/tmp/diag_1779541283729.svg', write_to='/tmp/diag_1779541283729.png', output_width=900, output_height=380)" 2>/dev/null]

5.1 Internet → EC2 Flow (per region)

- User sends HTTP (port 80) or SSH (port 22) request to the instance's public IP
- Traffic hits the Internet Gateway of the respective region
- IGW forwards to the subnet; Route Table has 0.0.0.0/0 → IGW, so it reaches the subnet
- Security Group evaluates the inbound rule — port 22 and 80 are allowed from 0.0.0.0/0
- EC2 instance receives the request and Apache serves the response (port 80) or SSH shell (port 22)

5.2 EC2 Primary ↔ EC2 Secondary (VPC Peering)

- Primary EC2 (10.0.x.x) sends a packet destined for 10.1.x.x
- Primary Route Table matches 10.1.0.0/16 → vpc-peering-connection-id
- AWS routes the packet over the private peering backbone (no internet)
- Secondary Route Table on the other end has 10.0.0.0/16 → vpc-peering-connection-id for the return path
- Security Group on secondary EC2 allows ICMP (ping) and any other configured ports
- Private communication established — both instances can ping/curl each other using private IPs

6. VPC Peering — Request / Accept / Route Flow

[Diagram rendering unavailable: Command failed: python3 -c "import cairosvg; cairosvg.svg2png(url='/tmp/diag_1779541283763.svg', write_to='/tmp/diag_1779541283763.png', output_width=860, output_height=340)" 2>/dev/null]

7. Step-by-Step Implementation Procedure

Step 1 — Set up Terraform Providers with Aliases (providers.tf)

Two AWS provider blocks were created with aliases — primary (us-east-1) and secondary (us-west-2). This allows Terraform to manage resources in two different regions from a single configuration using the provider = aws.primary / aws.secondary argument on each resource.

```
provider "aws" { region = var.primary_region  alias = "primary" }
provider "aws" { region = var.secondary_region alias = "secondary" }
```

Step 2 — Generate SSH Key Pairs Per Region

SSH key pairs were generated locally using PowerShell/ssh-keygen and registered with AWS using aws ec2 import-key-pair. A separate key pair was created for each region since AWS key pairs are regional resources.

```
ssh-keygen -t rsa -b 2048 -f primary-key
ssh-keygen -t rsa -b 2048 -f secondary-key
```

```
aws ec2 import-key-pair --key-name primary-key --public-key-material fileb://primary-key.pub --region us-east-1
aws ec2 import-key-pair --key-name secondary-key --public-key-material fileb://secondary-key.pub --region us-west-2
```

Step 3 — Define Variables (variables.tf)

Input variables were defined for regions, VPC CIDRs, and instance type to keep the configuration reusable and avoid hardcoding values. Non-overlapping CIDRs (10.0.0.0/16 and 10.1.0.0/16) are required for VPC peering to work.

```
variable "primary_region" { default = "us-east-1" }
variable "secondary_region" { default = "us-west-2" }
variable "primary_vpc_cidr" { default = "10.0.0.0/16" }
variable "secondary_vpc_cidr" { default = "10.1.0.0/16" }
variable "instance_type" { default = "t3.micro" }
```

Step 4 — Create Data Sources (data.tf)

Data sources dynamically fetch: (1) available Availability Zones per region so AZ names are never hardcoded, and (2) the latest Amazon Linux 2023 AMI per region using a name filter — so the newest, most patched AMI is always used.

```
data "aws_availability_zones" "primary_az" { provider = aws.primary state = "available" }
data "aws_ami" "primary_ami" { provider = aws.primary most_recent = true owners = ["amazon"]
  filter { name = "name" values = ["al2023-ami-*-x86_64"] } }
```

Step 5 — Create VPCs with DNS Support (main.tf)

Two VPCs were provisioned — one per region. Both have `enable_dns_support` and `enable_dns_hostnames` set to true so AWS assigns DNS hostnames to instances, which is required for service discovery inside the VPC.

```
resource "aws_vpc" "primary_vpc" {
  provider          = aws.primary
  cidr_block        = var.primary_vpc_cidr # 10.0.0.0/16
  enable_dns_support = true
  enable_dns_hostnames = true
}
```

Step 6 — Create Public Subnets

One public subnet was created per VPC. The `availability_zone` is set from the data source (`names[0]`) to avoid hardcoding. `map_public_ip_on_launch = true` ensures every EC2 instance launched into the subnet automatically receives a public IP, enabling direct SSH and HTTP access.

```
resource "aws_subnet" "primary_subnet" {
  provider          = aws.primary
  vpc_id            = aws_vpc.primary_vpc.id
  availability_zone  = data.aws_availability_zones.primary_az.names[0]
  map_public_ip_on_launch = true
}
```

```
}
```

Step 7 — Create Internet Gateways

One IGW per VPC was created and attached. Without an IGW, instances in public subnets have no path to the internet — they cannot be reached via HTTP/SSH, and cannot download packages during boot (yum update/install in the user_data script would fail).

```
resource "aws_internet_gateway" "primary_igw" {
  provider = aws.primary
  vpc_id   = aws_vpc.primary_vpc.id
}
```

Step 8 — Create Route Tables with Default Internet Route

Custom route tables were created for each VPC with a default route (0.0.0.0/0) pointing to the respective IGW. This sends all non-local traffic to the internet. The peering routes are added later via aws_route resources (after peering is accepted).

```
resource "aws_route_table" "primary_route_table" {
  provider = aws.primary
  vpc_id   = aws_vpc.primary_vpc.id
  route { cidr_block = "0.0.0.0/0" gateway_id = aws_internet_gateway.primary_igw.id }
}
```

Step 9 — Associate Route Tables with Subnets

Route table associations link each custom route table to its subnet. Without this, the subnet uses the VPC's default route table which only has the local VPC route — no internet and no peering routes.

```
resource "aws_route_table_association" "primary_rta" {
  provider      = aws.primary
  subnet_id    = aws_subnet.primary_subnet.id
  route_table_id = aws_route_table.primary_route_table.id
}
```

Step 10 — Create VPC Peering Connection (Request)

A peering connection request was initiated from the primary provider, targeting the secondary VPC in the secondary region. auto_accept = false because cross-region peering requires the peer region to accept it separately — it cannot self-accept.

```
resource "aws_vpc_peering_connection" "vpc_peering_request" {
  provider      = aws.primary
  vpc_id        = aws_vpc.primary_vpc.id
  peer_vpc_id   = aws_vpc.secondary_vpc.id
  peer_region   = var.secondary_region
  auto_accept   = false
}
```

Step 11 — Accept the Peering Connection (Acceptor)

The secondary provider accepts the pending peering request using `aws_vpc_peering_connection_accepter`. `auto_accept = true` means Terraform automatically accepts it. Terraform handles the dependency — the acceptor references the peering connection by ID.

```
resource "aws_vpc_peering_connection_accepter" "secondary_acceptor" {
  provider           = aws.secondary
  vpc_peering_connection_id = aws_vpc_peering_connection.vpc_peering_request.id
  auto_accept       = true
}
```

Step 12 — Add Peering Routes (Bidirectional)

Two `aws_route` resources were created — one in each route table — to enable bidirectional traffic over the peering connection. Both use `depends_on =`

`[aws_vpc_peering_connection_accepter.secondary_acceptor]` to ensure the connection is accepted before routes are added (otherwise Terraform would try to create routes over a non-active connection).

```
# Primary → Secondary
resource "aws_route" "primary_to_secondary" {
  provider           = aws.primary
  route_table_id     = aws_route_table.primary_route_table.id
  destination_cidr_block = var.secondary_vpc_cidr # 10.1.0.0/16
  vpc_peering_connection_id = aws_vpc_peering_connection.vpc_peering_request.id
  depends_on         = [aws_vpc_peering_connection_accepter.secondary_acceptor]
}

# Secondary → Primary (same pattern with aws.secondary provider)
```

Step 13 — Create Security Groups (Port 22, 80, ICMP)

One security group per VPC was created with ingress rules for SSH (22), HTTP (80), and ICMP (ping). ICMP is important here — it allows using ping to verify that the VPC peering connection is working (i.e., primary EC2 can ping secondary's private IP and vice versa). All egress traffic is allowed.

```
ingress { from_port=22 to_port=22 protocol="tcp" cidr_blocks=["0.0.0.0/0"] }
ingress { from_port=80 to_port=80 protocol="tcp" cidr_blocks=["0.0.0.0/0"] }
ingress { from_port=-1 to_port=-1 protocol="icmp" cidr_blocks=["0.0.0.0/0"] }
egress { from_port=0 to_port=0 protocol="-1" cidr_blocks=["0.0.0.0/0"] }
```

Step 14 — Write User Data Scripts (primary.sh / secondary.sh)

Shell scripts were written for each instance. On first boot, EC2 runs the `user_data` script which updates packages, installs Apache HTTPD, enables it as a system service, and writes a unique HTML page. The different page titles (Primary / Secondary) make it easy to verify which region is responding.

```
#!/bin/bash
yum update -y && yum install -y httpd
systemctl enable httpd && systemctl start httpd
echo "<h1>Primary Region Apache Server</h1>" > /var/www/html/index.html
```

Step 15 — Launch EC2 Instances

Two EC2 instances were launched — one per region — using the dynamically fetched AMI ID, the subnet, the security group, the SSH key pair, and the user_data shell script. Terraform handles the correct deployment order via implicit dependencies.

```
resource "aws_instance" "primary_instance" {
  provider          = aws.primary
  ami               = data.aws_ami.primary_ami.id
  instance_type     = var.instance_type    # t3.micro
  subnet_id         = aws_subnet.primary_subnet.id
  vpc_security_group_ids = [aws_security_group.primary_sg.id]
  key_name           = "primary-key"
  user_data         = file("./primary.sh")
}
```

Step 16 — Define Outputs

Output values were configured to display the public IPs of both instances after terraform apply. This provides immediate access to SSH in or open the Apache page in a browser.

```
output "primary_public_ip" { value = aws_instance.primary_instance.public_ip }
output "secondary_public_ip" { value = aws_instance.secondary_instance.public_ip }
```

Step 17 — Deploy with Terraform

```
terraform init
terraform validate
terraform plan
terraform apply
```

Terraform provisions all 20+ resources in dependency order. After apply, both public IPs are printed.

Step 18 — Verify the Setup

Verification was done in three ways:

- Browser: `http://<primary_public_ip>` → 'Primary Region Apache Server'
- Browser: `http://<secondary_public_ip>` → 'Secondary Region Apache Server'
- SSH into primary EC2 and ping secondary's private IP (10.1.x.x) — confirms peering works

```
ssh -i primary-key ec2-user@<primary_public_ip>
ping 10.1.x.x    # should get replies from secondary EC2
```

8. CIDR & Routing Reference Table

Network	Details
Primary VPC CIDR	10.0.0.0/16

Secondary VPC CIDR	10.1.0.0/16
Primary Subnet	10.0.0.0/16 (same CIDR — fills VPC)
Secondary Subnet	10.1.0.0/16 (same CIDR — fills VPC)
Primary RT route 1	0.0.0.0/0 → Primary IGW (internet)
Primary RT route 2	10.1.0.0/16 → vpc-peering-id (private)
Secondary RT route 1	0.0.0.0/0 → Secondary IGW (internet)
Secondary RT route 2	10.0.0.0/16 → vpc-peering-id (private)

9. Terraform Commands Reference

Initialize (download providers):

```
terraform init
```

Validate syntax:

```
terraform validate
```

Preview changes:

```
terraform plan
```

Deploy infrastructure:

```
terraform apply
```

Destroy all resources:

```
terraform destroy
```

Show current state:

```
terraform show
```

Refresh state from AWS:

```
terraform refresh
```

10. Project Summary — Step by Step

Complete Summary of What Was Done

1. Set up providers.tf with two AWS provider aliases — aws.primary (us-east-1) and aws.secondary (us-west-2) — to manage two regions from a single Terraform configuration.
2. Generated two SSH key pairs (primary-key and secondary-key) locally and registered each with the correct AWS region using aws ec2 import-key-pair.
3. Defined variables for regions, VPC CIDRs (10.0.0.0/16 and 10.1.0.0/16, non-overlapping), and instance type (t3.micro).
4. Created data sources to dynamically fetch available AZs and the latest Amazon Linux 2023 AMI per region — no hardcoded AZ names or AMI IDs.
5. Provisioned two VPCs with DNS support and hostname resolution enabled — one per region, each with a unique non-overlapping CIDR.

6. Created two public subnets (one per VPC) with `map_public_ip_on_launch = true` so instances get public IPs automatically for SSH/HTTP access.
7. Attached one Internet Gateway per VPC to enable inbound/outbound internet traffic for the public instances.
8. Created custom route tables with a default internet route (`0.0.0.0/0 → IGW`) and associated them to the subnets.
9. Initiated a cross-region VPC Peering Connection request from primary (`auto_accept = false`) and accepted it in the secondary region using `aws_vpc_peering_connection_accepter` (`auto_accept = true`).
10. Added bidirectional peering routes: `10.1.0.0/16 → peering` in the primary route table, and `10.0.0.0/16 → peering` in the secondary route table — with `depends_on` to wait for peering acceptance.
11. Created security groups in both VPCs allowing SSH (22), HTTP (80), and ICMP (ping) ingress from anywhere, and all egress traffic.
12. Wrote `user_data` shell scripts (`primary.sh`, `secondary.sh`) that auto-install and start Apache HTTPD on first boot and write a unique HTML page per region.
13. Launched two EC2 instances (`t3.micro`, `AL2023`) — one per region — each attached to the correct subnet, security group, SSH key, and `user_data` script.
14. Defined outputs for both public IPs so they are displayed immediately after terraform apply.
15. Deployed with `terraform init → validate → plan → apply` and verified: Apache pages load over HTTP, and private cross-region pinging confirms VPC Peering is fully operational.

Result: Two EC2 instances in different AWS regions, each running Apache, connected privately via VPC Peering — fully provisioned and reproducible using Terraform IaC in a single terraform apply.